# Protocol Document for Large Language Models for Architectural Analysis and Reasoning Support to Design Self-Adaptive Software Systems - A Systematic Literature Review

(Protocol Document)

## 1. Introduction

Modern software systems are increasingly required to operate in complex, heterogeneous, and dynamic environments while satisfying multiple, often competing, quality requirements. Rapid advances in technology, combined with growing system scale and openness, significantly increase the complexity of software architectures and enlarge the architectural design space. As a result, software architects face substantial cognitive load when reasoning alternative architectural solutions, evaluating trade-offs among quality attributes, and making design decisions under uncertainty. This complexity is further amplified by the scope and depth of architectural analysis and reasoning required across different stages of the system lifecycle, from requirements of interpretation to architectural decision-making.

The challenges become even more pronounced for systems that are expected to operate in dynamic and uncertain operational environments, where workloads, resources, user needs, and contextual conditions may change at runtime. Self-Adaptive Software Systems (SASS) have been proposed to cope with such uncertainty by enabling systems to monitor their execution context, analyze changes, and autonomously adapt their behavior at runtime. SASS constitute a well-established research area, offering mature mechanisms such as feedback control loops, runtime reconfiguration, and architectural adaptation techniques to maintain system goals related to reliability, performance, scalability, and availability under changing conditions. Architectural reasoning plays a central role in SASS, as adaptation decisions are typically guided by architectural models, quality trade-offs, and design rationale.

Recent advances in Large Language Models (LLMs) introduce new possibilities for supporting architectural reasoning and decision-making. Due to their capabilities in abstraction, pattern recognition, natural language understanding, and knowledge synthesis, LLMs have emerged as promising candidates for assisting software architects in tasks such as architectural analysis, trade-off evaluation, context interpretation, and decision support. These capabilities suggest potential value for both design-time and runtime architectural reasoning in self-adaptive systems.

However, existing research on the use of LLMs in this context remains fragmented. Most studies to date focus either on LLM applications in software engineering more broadly or on self-adaptive systems without explicitly addressing architectural reasoning and design. The intersection of

LLM-supported architectural reasoning and the design of SASS is therefore underexplored, and there is limited consolidated knowledge regarding how LLMs are integrated architecturally, what roles they play in self-adaptation, and how their use is optimized and evaluated.

To address this gap, this study defines a Systematic Literature Review (SLR) protocol aimed at investigating the current state of research on the use of LLMs for architectural analysis and reasoning in the design of Self-Adaptive Software Systems. The objective of the review is to systematically identify, classify, and synthesize existing studies at this intersection, assess how LLMs are used to support architectural reasoning, identify prevailing techniques for optimization and evaluation, and highlight research gaps and emerging directions. Ultimately, this SLR seeks to provide a structured foundation for future research on architecture-centric integration of LLM-based reasoning into self-adaptive and context-aware software systems.

## 2. Research Questions

This study is guided by the following Research Questions (RQs):

- **RQ1:** What has been reported to date on the use of LLMs for architectural analysis and reasoning support in the design of SASS and their product lines?
  ***Rationale:*** This question maps the state of the art, clarifying how and where LLMs are being applied in architectural tasks, what problems they address, and what limitations or gaps exist.
- **RQ2:** What techniques are reported to optimize and evaluate LLMs for architectural analysis and reasoning support in the design of SASS?
  ***Rationale:*** LLM-supported architectural reasoning is useful if the models produce accurate, grounded, and trustworthy results, particularly in adaptive systems. Examining existing optimization and evaluation techniques helps determine whether current methods are sufficient for these needs and highlights areas where improvement is critical. This makes RQ2 essential for advancing reliable LLM use in SASS design.
- **RQ3: What** are the emerging trends in the use of LLMs for architectural analysis and reasoning support in the design of SASS?
  ***Rationale:*** Understanding emerging trends helps anticipate future innovations, shaping the development of new tools, frameworks, and methodologies aligned with evolving practices to design and develop SASS.

These RQs define the scope and direction of the review and guide all subsequent activities, including search, selection, and data extraction.

## 3. Research Method

This study follows the SLR methodology based on the guidelines proposed by Kitchenham et al. The methodology sets out how to do this consistently in terms of selection, evaluation, and synthesis.

## 4. Planning the Review

This section describes in detail how the review is planned and designed to ensure rigor, transparency, and reproducibility.

### 4.1 Identifying the Need for the Review

There is a research gap on the intersection of LLMs, software architecture design, and SASS. While there is research available on LLMs that is applied in SASS and SE tasks, their combined role is still unexplored and resulting in an immature domain. As a result, a SLR is essential for:

- Consolidation of existing research
- Assessment of current approaches
- Identification of limitations and challenges

### 4.2 Defining Research Questions

The guide for the entire process of review is based on the RQs that are defined in **Section 2.** They provide direction towards the data that needs to be collected, the evaluation of studies, and how the findings are synthesized.

### 4.3 Search Strategy

We have designed the search strategy to identify relevant studies in a comprehensive and systematic way so that biasness can be minimized.

#### 4.3.1 Keyword Derivation and Variant Mapping

The search terms used in this review were systematically derived from the Research Questions (RQ1–RQ3) by identifying a core set of **conceptual keywords**, including *LLMs*, *software architecture*, *architectural analysis and reasoning*, *SASS*, *product lines*, *optimization techniques*, *evaluation methods*, and *emerging research directions*. Given the emerging and interdisciplinary nature of the research area, these cores keywords are expressed heterogeneously across the literature. Therefore, for each primary keyword, we identified a set of **synonyms, variants, and closely related terms** as they appear in the included studies. These variants include, for example, alternative formulations such as *reasoning plan generation*, *decision justification*, *dynamic reconfiguration*, *agentic systems*, *hybrid architectures*, *retrieval-augmented generation*, and *adaptive workflows*. All identified keyword variants were documented in a supporting file named ‘’Keywords_and_Similar_Words.xlxs’’ in the Replication_Package to ensure coverage of

semantically equivalent terminology used across different research communities and application domains.

#### 4.3.2 Search Approach

There are three approaches combined in the search process:

- **Automatic search** for retrieval of studies using the predefined search strings
- **Manual search** to keep the relevant studies and filter out the irrelevant ones
- **Snowballing** is implemented in (forward and backward) way to find out the additional studies

The overall combination results in improved coverage of studies and reduces the risk of missing relevant studies.

#### 4.3.3 Data Sources

The following libraries/databases are used to extract the data:

- IEEE Xplore
- Scopus
- ACM Digital Library

The reason for selecting these libraries is because the high-quality publications about SE, architecture design, and intelligent systems are covered by them.

#### 4.3.4 Search String

We derived the search string by extracting the keywords from our RQs and combined them into following three categories:

- Terms related to LLMs
- Terms related to architectural design and reasoning
- Terms related to SASS

To form the final search string, the keywords are combined with Boolean operators. The details of search strings executed are as follows:

## General search string

( "large language model" OR "large language models" OR LLM OR LLMs OR
"generative AI" OR "generative Artificial Intelligence" OR "Gen AI" OR
"Gen Artificial Intelligence" OR "generative model" OR
"Transformer model" OR "Transformer models" OR

GPT OR "GPT-4" OR "GPT-3" OR ChatGPT OR BERT OR RoBERTa OR T5
OR PaLM OR LLaMA OR Claude)
AND
( "software architecture" OR "architecture design" OR "architectural design" OR
"architecture modeling" OR "architectural modeling" OR "architecture evaluation" OR
"architecture analysis" OR "architectural reasoning" OR "architecture decision-making" OR
"design rationale" OR "decision support" OR "architecture framework" OR
"architecture pattern" OR "architecture tactic" OR "trade-off analysis" OR
"quality attribute analysis" OR "design decision")
AND (adapt* OR self* OR autonom*)

**Note: This is the general query that we generated by extracting keywords from our Research Questions.**

## IEEE Xplore search string

(( "Document Title":"large language model" OR "Document Title":"large language models" OR "Document Title":"LLMs" OR "Document Title":LLM OR "Document Title":"generative AI" OR "Document Title":"generative Artificial Intelligence" OR "Document Title":"Gen AI" OR "Document Title":"Gen Artificial Intelligence" OR "Document Title":"generative model" OR "Document Title":"Transformer models" OR "Document Title":"Transformer model" OR "Document Title":GPT OR "Document Title":"GPT-4" OR "Document Title":"GPT-3" OR "Document Title":"ChatGPT" OR "Document Title":"BERT" OR "Document Title":"RoBERTa" OR "Document Title":"T5" OR "Document Title":"PaLM" OR "Document Title":"LLaMA" OR "Document Title":"Claude" OR "Abstract":"large language model" OR "Abstract":"large language models" OR "Abstract":"LLMs" OR "Abstract":LLM OR "Abstract":"generative AI" OR "Abstract":"generative Artificial Intelligence" OR "Abstract":"Gen AI" OR "Abstract":"Gen Artificial Intelligence" OR "Abstract":"generative model" OR "Abstract":"Transformer models" OR "Abstract":"Transformer model" OR "Abstract":GPT OR "Abstract":"GPT-4" OR "Abstract":"GPT-3" OR "Abstract":"ChatGPT" OR "Abstract":"BERT" OR "Abstract":"RoBERTa" OR "Abstract":"T5" OR "Abstract":"PaLM" OR "Abstract":"LLaMA" OR "Abstract":"Claude" ) AND ( "Document Title":"software architecture" OR "Document Title":"architecture design" OR "Document Title":"architectural design" OR "Document Title":"architecture modeling" OR "Document Title":"architectural modeling" OR "Document Title":"architecture evaluation" OR "Document Title":"architecture analysis" OR "Document Title":"architectural reasoning" OR "Document Title":"architecture decision-making" OR "Document Title":"design rationale" OR "Document Title":"decision support" OR "Document Title":"architecture framework" OR "Document Title":"architecture pattern" OR "Document Title":"architecture tactic" OR "Document Title":"trade-off analysis" OR "Document Title":"quality attribute analysis" OR "Document Title":"design decision" OR "Abstract":"software architecture" OR "Abstract":"architecture design" OR "Abstract":"architectural design" OR "Abstract":"architecture modeling" OR "Abstract":"architectural modeling" OR "Abstract":"architecture evaluation" OR "Abstract":"architecture analysis" OR "Abstract":"architectural reasoning" OR

"Abstract":"architecture decision-making" OR "Abstract":"design rationale" OR "Abstract":"decision support" OR "Abstract":"architecture framework" OR "Abstract":"architecture pattern" OR "Abstract":"architecture tactic" OR "Abstract":"trade-off analysis" OR "Abstract":"quality attribute analysis" OR "Abstract":"design decision" ) AND ( "Document Title":adapt* OR "Document Title":self* OR "Document Title":autonom* OR "Abstract":adapt* OR "Abstract":self* OR "Abstract":autonom* ))

### Scopus search string

( TITLE ( "large language model" OR "large language models" OR "LLMs" OR LLM OR "generative AI" OR "generative Artificial Intelligence" OR "Gen AI" OR "Gen Artificial Intelligence" OR "generative model" OR "Transformer models" OR "Transformer model" OR GPT OR "GPT-4" OR "GPT-3" OR "ChatGPT" OR "BERT" OR "RoBERTa" OR "T5" OR "PaLM" OR "LLaMA" OR "Claude" ) OR ABS ( "large language model" OR "large language models" OR "LLMs" OR LLM OR "generative AI" OR "generative Artificial Intelligence" OR "Gen AI" OR "Gen Artificial Intelligence" OR "generative model" OR "Transformer models" OR "Transformer model" OR GPT OR "GPT-4" OR "GPT-3" OR "ChatGPT" OR "BERT" OR "RoBERTa" OR "T5" OR "PaLM" OR "LLaMA" OR "Claude" ) ) AND ( TITLE ( "software architecture" OR "architecture design" OR "architectural design" OR "architecture modeling" OR "architectural modeling" OR "architecture evaluation" OR "architecture analysis" OR "architectural reasoning" OR "architecture decision-making" OR "design rationale" OR "decision support" OR "architecture framework" OR "architecture pattern" OR "architecture tactic" OR "trade-off analysis" OR "quality attribute analysis" OR "design decision" ) OR ABS ( "software architecture" OR "architecture design" OR "architectural design" OR "architecture modeling" OR "architectural modeling" OR "architecture evaluation" OR "architecture analysis" OR "architectural reasoning" OR "architecture decision-making" OR "design rationale" OR "decision support" OR "architecture framework" OR "architecture pattern" OR "architecture tactic" OR "trade-off analysis" OR "quality attribute analysis" OR "design decision" ) ) AND ( TITLE ( adapt* OR self* OR autonom* ) OR ABS ( adapt* OR self* OR autonom* ) )

## ACM/DL

(Title:"large language model" OR Title:"large language models" OR Title:"llms" OR Title:llm OR Title:"generative ai" OR Title:"generative artificial intelligence" OR Title:"gen ai" OR Title:"gen artificial intelligence" OR Title:"generative model" OR Title:"transformer models" OR Title:"transformer model" OR Title:gpt OR Title:"gpt-4" OR Title:"gpt-3" OR Title:"chatgpt" OR Title:"bert" OR Title:"roberta" OR Title:"t5" OR Title:"palm" OR Title:"llama" OR Title:"claude" OR Abstract:"large language model" OR Abstract:"large language models" OR Abstract:"llms" OR Abstract:llm OR Abstract:"generative ai" OR Abstract:"generative artificial intelligence" OR Abstract:"gen ai" OR Abstract:"gen artificial intelligence" OR Abstract:"generative model" OR Abstract:"transformer models" OR Abstract:"transformer model" OR Abstract:gpt OR Abstract:"gpt-4" OR Abstract:"gpt-3" OR Abstract:"chatgpt" OR Abstract:"bert" OR Abstract:"roberta" OR Abstract:"t5" OR Abstract:"palm" OR Abstract:"llama" OR Abstract:"claude")

AND
(Title:"software architecture" OR Title:"architecture design" OR Title:"architectural design" OR Title:"architecture modeling" OR Title:"architectural modeling" OR Title:"architecture evaluation" OR Title:"architecture analysis" OR Title:"architectural reasoning" OR Title:"architecture decision-making" OR Title:"design rationale" OR Title:"decision support" OR Title:"architecture framework" OR Title:"architecture pattern" OR Title:"architecture tactic" OR Title:"trade-off analysis" OR Title:"quality attribute analysis" OR Title:"design decision" OR Abstract:"software architecture" OR Abstract:"architecture design" OR Abstract:"architectural design" OR Abstract:"architecture modeling" OR Abstract:"architectural modeling" OR Abstract:"architecture evaluation" OR Abstract:"architecture analysis" OR Abstract:"architectural reasoning" OR Abstract:"architecture decision-making" OR Abstract:"design rationale" OR Abstract:"decision support" OR Abstract:"architecture framework" OR Abstract:"architecture pattern" OR Abstract:"architecture tactic" OR Abstract:"trade-off analysis" OR Abstract:"quality attribute analysis" OR Abstract:"design decision")
AND
(Title:adapt* OR Title:self* OR Title:autonom* OR Abstract:adapt* OR Abstract:self* OR Abstract:autonom*)

#### 4.3.5 Search Validation

To validate the adequacy and coverage of the search string, we applied the **Quasi Gold Standard (QGS)** method, a well-established approach for assessing search string effectiveness in systematic literature reviews. The QGS method evaluates whether a predefined set of known relevant studies (QGS articles) can be retrieved using the proposed search strategy, thereby providing confidence in its recall capability. Given that research at the intersection of **LLMs**, **software architecture**, and **SASS** is still **immature and emerging**, we observed that the studies do not explicitly cover all three dimensions simultaneously. To address this challenge and avoid prematurely excluding relevant work, we adopted a **pairwise concept validation approach** when applying the QGS. This approach ensures coverage of all relevant conceptual dimensions of LLMs, software architecture, and self-adaptation even when a study addresses only a subset of these dimensions explicitly.

Specifically, for each QGS article, we executed the relevant **pairwise combinations of keyword groups** from the overall search string, rather than requiring all keyword groups to be present simultaneously. For example, for QGS articles primarily focusing on **LLMs and software architecture design**, only the LLM-related and architecture-related components of the search string were applied. Similarly, for QGS articles focusing on **LLMs and self-adaptation**, the LLM-related and self-adaptation-related components were used. This strategy reflects the fragmented nature of the current literature and allows validation of the search string's ability to retrieve studies spanning different conceptual intersections.

**Table 1** presents the selected QGS articles along with the corresponding keyword groups used for their retrieval, indicating which conceptual dimensions of LLMs, software architecture, and self-adaptation are explicitly addressed by each article. The successful retrieval of all QGS articles through at least one relevant pairwise keyword combination provides confidence that the search string sufficiently captures the breadth of the research landscape while remaining sensitive to the emerging and heterogeneous nature of the field. The details of QGS articles are also given in Master_Index.xlxs with file name '' Quasi_Gold_Standard_Articles''.

**Table 1: QGS Articles and the Keywords Parts used for Article ~~Extraction~~**

| QGS Article | LLM Keywords | Architecture Keywords | Self-adaptation Keywords |
|---|---|---|---|
| Can LLMs generate architectural design decisions? —An exploratory empirical study [1] | ✓ | ✓ | ✗ |
| Reimagining self-adaptation in the age of large language models [2] | ✓ | ✗ | ✓ |
| Knowledge-Based Multi-Agent Framework for Automated Software Architecture Design [3] | ✓ | ✓ | ✗ |
| Self-Adaptive Large Language Model (LLM)-Based Multiagent Systems [4] | ✓ | ✗ | ✓ |
| Leveraging Self-Adaptive Systems and Generative AI for Personalizing Educational Serious Games: Architecture and Future Challenges [5] | ✓ | ✗ | ✓ |

#### 4.3.6 Time Span

Corresponding to the rapid development and emergence of LLMs, our search covers the studies published between **January 2018 to October 2025**.

### 4.4 Study Selection Criteria

We have included/excluded the studies based on some criteria's given as follows:

#### 4.4.1 Inclusion Criteria

Studies are included if they ensure that the selected articles contain the intersection of LLMs, SASS, and software architecture design so that the relevance of our research objective is maintained.

### 4.4.2 Exclusion Criteria

Studies are excluded if:

- There are categories other than conferences and journals
- They are not written in English, since it's a primary language for research community
- If the articles are shorter than 5 pages in length

The criteria ensure relevance, quality, and sufficient detail.

## 4.5 Primary Study Selection Procedure

We have performed the primary study selection in multiple steps:

1. Removal of duplicate studies
2. Screening of title and abstract
3. Full text review **(only if needed)**
4. Conflicts were resolved among researchers with mutual discussion

The consistency and reduction of biasness is done with this multi-step process.

## 4.6 Data Extraction

The data extraction is performed following the steps given below:

### 4.6.1 Data Items

We extracted the data using predefined data items (D1-D18), they are grouped into:

- Documentation
- Quality assessment
- RQ1-related data
- RQ2-related data
- RQ3-related data

The file containing data items is given in Master_Index.xlxs with name ''Data_Extraction_Items.xlxs''.

### 4.6.2 Data Extraction Process

Data is extracted systematically and then recorded in a structured format (spreadsheets) to ensure traceability and consistency. The link to the study data is given here: Replication_Package.

### 4.7 Quality Assessment

A predefined criteria was set to evaluate the quality of primary studies:

- Score of criteria was from **0 to 2**
- Total range of scores was from **0 to 12**

Quality assessment helped in evaluation of studies in terms of their reliability and supported the weighted interpretation of results. The file Reasons_for_Assigned_Quality_Scores.pdf provides a transparent and traceable justification for all quality assessment decisions. This file documents, for each of the three primary studies, the rationale behind the assigned scores for all six quality criteria (Q1–Q6), including problem definition, problem context, research design, contributions, insights, and limitations. For each criterion, the file explicitly indicates the exact sections, pages, figures, or paragraphs in the original publications that support the assigned score, thereby ensuring consistency, auditability, and replicability of the quality assessment process. Where limitations are only implicitly discussed, this is clearly noted and reflected in the corresponding score. By linking each quality score to concrete evidence in the source studies, this artifact strengthens the methodological rigor of the review and allows independent researchers to verify, reproduce, or reassess the quality evaluation if needed.

## 5. Conducting the Review

The review is conducted in the following steps:

### 5.1 Conducting Search

The predefined search string is executed among selected libraries in their specific formats (given in Section 4.3.3) to extract the relevant studies.

### 5.2 Selecting Primary Studies

To identify the primary studies, inclusion/exclusion criteria are applied (given in Section 4.4).

### 5.3 Data Analysis

When we executed the search strings in all three libraries, we got the following results:

We selected the following three libraries: ACM Digital Library, IEEE Xplore, and Scopus. The combination of three libraries was strategically used to minimize the risk of missing relevant studies, thereby strengthening the completeness and reliability of the evidence base. As the search engines used by these digital libraries differ in their query syntax, the search string was specialized and adapted for each library. Without using any filters, the search string produced **455** Scopus results, **20** ACM Digital Library results, and **75** IEEE Xplore results. We obtained 61 results from IEEE Xplore, **19** from ACM Digital Library, and **196** from Scopus after applying the filters (including conferences, journals, years between **2018 to October 2025**, and topics unique to

computer science). So, the final number of articles was **276**. We next applied the inclusion/exclusion criteria to these articles and removed duplicates. **58** duplicates were found and **218** were the unique articles resulting in a final number of **03** primary articles. Here is the list of 03 identified primary articles:

| Title | DOI |
|---|---|
| **(S1):** An Architecture for Integrating Large Language Models with Digital Twins and Automation Systems [6] | 10.1109/ETFA65518.2025.11205636 |
| **(S2):** Improving Adaptability in Optimization-Based Decision Support Systems Through Large Language Models [7] | 10.1109/ACCESS.2025.3601518 |
| (**S3):** An Adaptive Multi-Agent LLM-Based Clinical Decision Support System Integrating Biomedical RAG and Web Intelligence [8] | 10.1109/ACCESS.2025.3613340 |

The PDF files of three primary studies are also given in Replication_Package.

## 5.4 Overview of Identified Studies and Scope Extension

During the review process, the initial intention was to focus exclusively on studies addressing the use of LLMs for architectural analysis and reasoning in the design of SASS. However, despite a broad and systematic search across major digital libraries, only **three primary studies** met all inclusion and quality criteria. This limited number of studies highlight that research at the intersection of LLMs, software architecture, and self-adaptive systems is still at an early and fragmented stage. To better contextualize these findings and avoid drawing conclusions from an overly narrow evidence base, the scope of the study was expanded to include research on **LLM-supported software architecture design more broadly**, even when not explicitly focused on self-adaptation. This scope of expansion enabled the identification of **64 additional architecture-focused studies** that investigate LLMs for architectural decision support, trade-off analysis, and design assistance. Although these studies do not explicitly address SASS, they provide valuable architectural insights, techniques, and patterns that can inform future integration of LLM-based reasoning into self-adaptive and CASA systems. Accordingly, the expanded evidence base serves as a foundation for identifying trends, gaps, and opportunities for advancing architecture-centric LLM support in self-adaptive software systems.

## 5.5 Replication Package and Study Artifacts

The accompanying file named Master_Index constitutes the full set of study artifacts produced during the execution of the Systematic Literature Review. It consolidates all intermediate and results across multiple structured sub-sheets. Specifically, for example; it includes: (i) the complete **data extraction schema (D1–D18)** and extracted data for the three primary studies, covering bibliographic metadata, LLM models used, architectural activities supported, lifecycle phases, roles of LLMs, optimization and evaluation techniques, datasets, metrics, limitations, emerging trends, gaps, and future research directions; (ii) the complete results of the **automated searches** conducted in ACM Digital Library, IEEE Xplore, and Scopus, with explicit inclusion and

exclusion decisions based on predefined criteria; (iii) detailed logs of **duplicate identification and removal** across all libraries, documenting the transition from raw search hits to unique candidate studies; (iv) the resulting set of **unique articles after deduplication**, forming the basis for screening; (v) the definition and validation of the **QGS** articles, including the specific keyword subsets used to retrieve each QGS paper; (vi) comprehensive records of **forward and backward snowballing**, including cited and citing articles, accessibility checks, inclusion decisions, and reasons for exclusion; (vii) expanded file containing 64 results for the broader "LLMs and Software Architecture" scope used for contextualization with forward and backward snowballing results; and (viii) the final set of **three primary studies** selected for in-depth analysis, with explicit justifications for inclusion and architectural relevance to SASS. Together, these sub-sheets provide full transparency and traceability of the search, selection, validation, and analysis processes, enabling replication and extension of the review.

## 6. Reporting the Review Results

The results addressing RQ1–RQ3 collectively demonstrate that the use of LLMs for architectural analysis and reasoning in SASS remains **nascent and fragmented**. From an initial set of 276 potentially relevant studies, only **three primary studies** satisfied all inclusion and quality criteria, underscoring the early stage of research at the intersection of LLMs, software architecture, and self-adaptation. This limited evidence base indicates that, while interest in applying LLMs to adaptive and context-aware systems is growing, architecture-centric investigations are still rare and largely exploratory.

Across the identified studies, LLMs are consistently positioned **not as first-class architectural elements**, but as **augmentative reasoning components** integrated into broader system architectures. Their primary role is to assist with architectural reasoning tasks such as interpreting contextual information, supporting decision-making, exploring design alternatives, and generating architectural rationale rather than directly governing system adaptation. These contributions occur at both design time and runtime, most commonly within the *Analyze* and *Plan* phases of adaptation, while execution and adaptation control remain external to the LLMs themselves. Architecturally, the integration of LLMs follows **hybrid patterns**, where they are combined with complementary mechanisms such as digital twins, optimization engines, domain models, and knowledge bases, often structured as multi-agent or layered reasoning pipelines aligned with feedback-control loops.

To improve reasoning quality, reliability, and trustworthiness, the reviewed studies employ a range of optimization strategies. Common approaches include **retrieval-augmented grounding** to anchor LLM outputs in domain-specific knowledge, **structured and few-shot prompting** to constrain reasoning behavior, and **multi-agent decomposition** to distribute complex reasoning

tasks across specialized agents. Several studies also incorporate **iterative or feedback-driven refinement mechanisms**, allowing LLM-based reasoning to be adjusted in response to confidence assessments or evolving system context. Despite these techniques, evaluation practices remain **heterogeneous and largely non-standardized**. The studies primarily rely on case studies, simulations, controlled domain-specific experiments, or qualitative expert assessments, with little emphasis on architecture-aware benchmarks, formal verification, or long-term measures such as adaptation robustness, scalability, or architectural resilience. This lack of consistent and architecture-centric evaluation limits comparability across studies and hinders cumulative progress.

The results further reveal a set of emerging trends and persistent gaps. On the one hand, there is a clear movement toward **hybrid and grounded architectural solutions**, increased adoption of **agentic and multi-agent reasoning**, and greater emphasis on grounding, explainability, and human-in-the-loop interaction. On the other hand, the studies consistently expose significant shortcomings, including the absence of **architecture-centric integration frameworks**, incomplete coverage of the **full MAPE-K loop**, limited attention to **variability management and product-line support**, and a lack of **formal assurance and trustworthiness mechanisms** at the architectural level. Overall, while the field is progressing toward more structured uses of LLMs in adaptive contexts, the results indicate that research remains in an early stage and that substantial work is required to achieve systematic, rigorous, and architecture-driven integration of LLM-based reasoning into self-adaptive software systems.

## 7. Threats to Validity

This section discusses the main threats to the validity of this initial systematic litrature, and the measures taken to mitigate them. Following established guidelines for secondary studies, we organize the threats around search, selection, data extraction, synthesis, and external validity. These threats are particularly relevant given the novelty of the research area and the small number of primary studies identified.

**Search Validity** concerns whether the search strategy successfully captured relevant studies. Terminology in the LLM and SASS domains evolves rapidly, and relevant work may use alternative expressions (e.g., “AI agents”, “reasoning copilot”). To mitigate this, we derived keywords directly from the RQs, validated the search string using the QGS method, refined terminology through pilot searches, and complemented the automatic search with manual screening and backward–forward snowballing.

**Selection Validity** refers to the risk of incorrectly including or excluding studies. Because inclusion required interpreting whether a study addressed the intersection of LLMs, software

architecture, and SASS, subjective judgment could introduce bias. We mitigated this threat by relying on a predefined protocol with explicit inclusion and exclusion criteria, conducting multi-stage screening, and resolving uncertainties collaboratively among the authors.

**Data Extraction Validity** concerns the accuracy and consistency of the extracted data. Misinterpretation may arise when identifying architectural roles, optimization techniques, or evaluation of details. To reduce this risk, at least two authors independently reviewed extracted items, resolved discrepancies by consensus, and trace all extracted information back to the original studies.

**Synthesis Validity** concerns the risk of bias during interpretation and integration of evidence. With only three primary studies, subjective interpretation could influence the identification of architectural roles, optimization mechanisms, and research trends. We mitigated this by grounding the synthesis strictly in predefined data extraction items and by jointly reviewing synthesized findings to ensure traceability and consistency.

**External Validity** concerns the generalizability of the findings. The three included studies span distinct domains (clinical decision support, industrial automation, and optimization-based DSS), which may limit generalizability to other SASS context. The small evidence base may also reflect publication bias, as negative or inconclusive findings are less frequently reported. To address this, we interpret the results as preliminary indicators of emerging trends, gaps, and re- search opportunities rather than definitive conclusions about mature practices in LLM-supported architectural reasoning for SASS.

## References


1. Dhar, Rudra, Karthik Vaidhyanathan, and Vasudeva Varma. "Can llms generate architectural design decisions?-an exploratory empirical study." In *2024 IEEE 21st International Conference on Software Architecture (ICSA)*, pp. 79-89. IEEE, 2024.
2. Donakanti, Raghav, Prakhar Jain, Shubham Kulkarni, and Karthik Vaidhyanathan. "Reimagining self-adaptation in the age of large language models." In *2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C)*, pp. 171-174. IEEE, 2024.
3. Zhang, Yiran, Ruiyin Li, Peng Liang, Weisong Sun, and Yang Liu. "Knowledge-based multi-agent framework for automated software architecture design." In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, pp. 530-534. 2025.
4. Nascimento, Nathalia, Paulo Alencar, and Donald Cowan. "Self-adaptive large language model (llm)-based multiagent systems." In *2023 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C)*, pp. 104-109. IEEE, 2023.
5. Bucchiarone, Antonio, Federico Bonetti, and Enes Yigitbas. "Leveraging Self-Adaptive Systems and Generative AI for Personalizing Educational Serious Games: Architecture and Future Challenges." In *2025 IEEE/ACM 20th Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*, pp. 103-110. IEEE, 2025.
6. Xia, Yuchen, Nasser Jazdi, and Michael Weyrich. "An Architecture for Integrating Large Language Models with Digital Twins and Automation Systems." In *2025 IEEE 30th International Conference on Emerging Technologies and Factory Automation (ETFA)*, pp. 1-8. IEEE, 2025.
7. Ghiani, Gianpaolo, Emanuele Manni, and Sandro Zacchino. "Improving Adaptability in Optimization-Based Decision Support Systems Through Large Language Models." *IEEE Access* (2025).
8. Öğdü, Çağatay Umut, Kübra Arslanoğlu, and Mehmet Karaköse. "An adaptive multi-agent llm-based clinical decision support system integrating biomedical rag and web intelligence." *IEEE Access* (2025).